MSDM5002 Fall 2026

Final project

This is a group project. You are going to write an interactive program for a variant of the classic game "five-in-a-row" (aka "gomoku" and "wuziqi"), in which you can play against a computer that moves according to your strategy.

1. Rules of the game

Our variant of five-in-a-row uses a rectangular grid board with $N_R = 17$ rows and $N_C = 13$ columns (Figure 1). Its gridlines are **randomly initialized** according to two parameters: horizontal connectivity $C_H = 0.85$ and vertical connectivity $C_V = 0.7$. These are the probabilities for two points to be horizontally or vertically connected, and a line is drawn between two horizontally or vertically connected grid points. Although no diagonal lines are drawn, the points are always diagonally connected (in both $\diagup$ and $\diagdown$ directions).

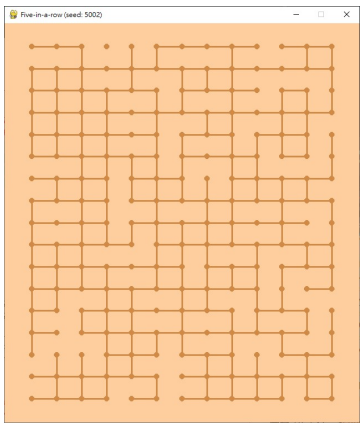


Figure 1 A game window with a randomly initialized board.

Like the standard game, two players (Black and White) alternately place their "**stones**" at any empty point. Black moves first.

- When an unbroken line of **exactly five** stones of the same colour is formed, that player wins the game.
- When a line connecting more than five stones of the same colour is formed, it is not counted as a win, and the game continues.

– When there are no more empty points, the game ends in a draw, but White is considered to have won because Black enjoys an advantage over White in the game by moving first.

Figures 2 to 5 illustrate the rules on a board with $N_R = 10$ rows and $N_C = 10$ columns.

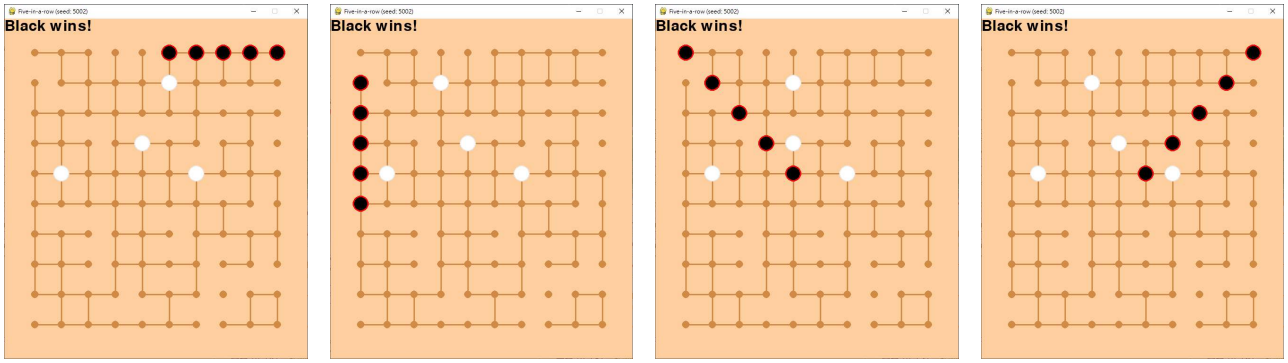


Figure 2 Black wins in the above games.

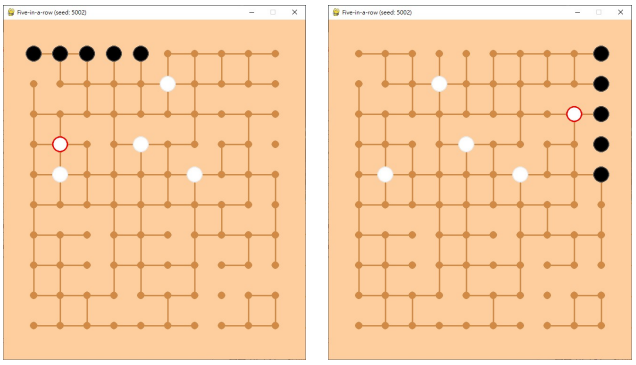


Figure 3 Black does not win in the above games because there are gaps between the five black stones.

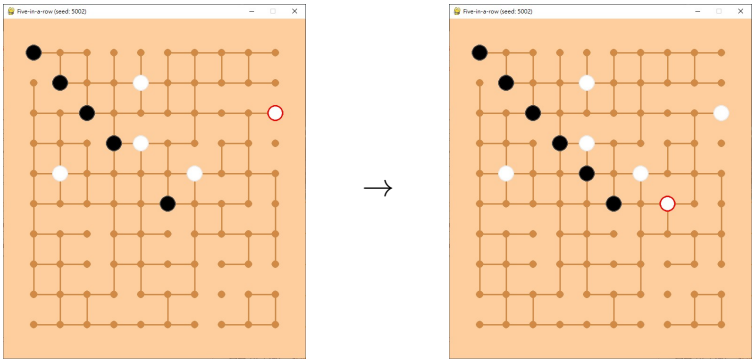


Figure 4 Black does not win in the above game because more than five black stones are connected.

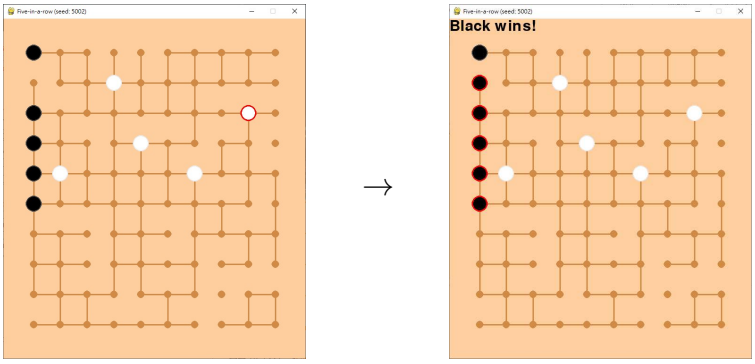


Figure 5 Black wins in the above game because only five black stones are connected.

2. Representation of the game

Let the top left corner of the board be its origin $(0, 0)$. Accordingly, the row number r is in $[0, N_R)$ and counts from top to bottom, while the column index c is in $[0, N_C)$ and counts from left to right. A state of a game can then be represented by a tuple `state=(board, edgeH, edgeV)` consisting of an $N_R \times N_C$ integer array `board`, an $N_R \times (N_C - 1)$ Boolean array `edgeH`, and an $(N_R - 1) \times N_C$ Boolean array `edgeV`.

- If `board[r, c] = +1`, the point (r, c) is occupied by a black stone.
- If `board[r, c] = -1`, the point (r, c) is occupied by a white stone.
- If `board[r, c] = 0`, the point (r, c) is empty.
- If `edgeH[r, c] = True`, the points (r, c) and $(r, c + 1)$ are (horizontally) connected.
- If `edgeH[r, c] = False`, the points (r, c) and $(r, c + 1)$ are not connected.
- If `edgeV[r, c] = True`, the points (r, c) and $(r + 1, c)$ are (vertically) connected.
- If `edgeV[r, c] = False`, the points (r, c) and $(r + 1, c)$ are not connected.

For example, Figure 6 shows the final state of a game with $N_R = N_C = 5$.

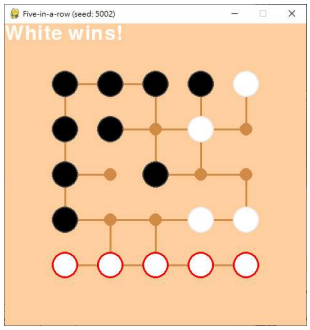


Figure 6 White wins in the above game.

This final state is represented by a tuple `state=(board, edgeH, edgeV)`, where `board`, `edgeH.astype(int)`, and `edgeV.astype(int)` are respectively equivalent to matrices

$$\begin{pmatrix} +1 & +1 & +1 & +1 & -1 \\ +1 & +1 & 0 & -1 & 0 \\ +1 & 0 & + & 0 & 0 \\ +1 & 0 & 0 & -1 & -1 \\ -1 & -1 & -1 & -1 & -1 \end{pmatrix}, \quad \begin{pmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix}, \text{ and } \begin{pmatrix} 1 & 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 \end{pmatrix}.$$

3. Tasks and requirements

Download `project_skeleton.py` and use it as the skeleton code. Various constants and functions have been defined for you, and they should not be changed (except `SEED`). Do not hardcode the constants since they may be modified during tests. You are only allowed to import Pygame, NumPy, and `time` into this program, where Pygame is used to create the GUI of the game. (If you do not have Pygame on your computer, you should install it first.)

Using the skeleton code, you should complete three functions `main(human_color)`, `best_move(state, color)`, and `check_result(state)` so that the program can do the following tasks. (As explained above, `state` is a tuple of arrays.)

(a) A game window pops up when `main(human_color)` is executed, showing a randomly initialized board. You (a human) and the computer will take turns to place stones. Remember Black moves first.

- If `human_color = +1`, you are Black, and the computer is White.
- If `human_color = -1`, you are White, and the computer is Black.
- Otherwise, the window is closed. [See (h) for an optional task.]

(b) When you click at a valid and empty position during your turns, a stone of your colour is placed at the corresponding grid point.

(c) During the computer's turns, it places a stone of its colour at the point returned by `best_move(state, color)`, which uses your strategy to determine the best point to place a stone.

- If `color = +1`, the stone to be placed is black.
- If `color = -1`, the stone to be placed is white.

Your strategy must be related to **Monte Carlo tree search** (MCTS), but you are allowed and also encouraged to incorporate it with other tactics (like occupying a specific point due to the configuration of the board).

This function should spend at most five seconds per move. Although there are no consequences for minor violations, the program will be considered frozen if a move takes too much time.

(d) When a stone is placed, it is drawn in the window with a highlighted border. The border is kept until a new stone is placed (by either player). As soon as the new stone appears, the previously highlighted stone is drawn again to remove the border.

(e) After each move, `check_result(state)` is called to check if the game has ended. If so, the game result is printed in the window ("Black wins!", "White wins!", or "Draw!"). If there is a winner, the five stones in the winning line are highlighted.

(f) The window is locked once the game ends, and your clicks can no longer change the board.

(g) You can close the window at any time (even before the game ends).

(h) **Optional task.** If `abs(human_color) != 1`, instead of closing the window, you may show a game between the computer and itself (like an animation). This helps you verify if your entire program runs correctly.

4. Submissions and group competition

Again, this is a group project—communication skills are as important as programming skills—so you will work in a group of 4–5 people (i.e. with 3–4 groupmates). The class is divided into 16 groups.

(a) **Program (35%).** Each group has to submit a single `.py` file as the game program. You must build it upon the skeleton code, and you should only modify the three required functions.

(b) **Report (35%).** You have to submit an individual PDF report with around 3–5 pages. It should include (i) your understanding of MCTS, (ii) an explanation of your group's strategy, (iii) your contribution to the group, (iv) the division of labour in your group, and (v) a peer evaluation table.

(c) **Peer evaluation (15%).** In the peer evaluation table, please give each of your groupmates a participation score from 0 to 5. The average score given to you by your groupmates will account for 15% of your total score in the project. If your report does not include this table, you will lose this 15% entirely.

(d) **Competition (15%).** During the exam period, a **knockout competition** will be held among the groups. The function `best_move(state, color)` will be imported from each group's program to a master program so that the groups can compete against each other using their strategies. Hence, please make sure that your function only depends on what is provided by the skeleton code.

- Each match has two rounds, and two groups take turns playing as Black and White. Each round uses a different seed. If a round ends in a draw, White is considered to have won that round. A group wins a match if it wins both rounds. If both groups win one round each, the group that spends less thinking time in total wins the match.
- A group gets 1% per winning round. If a group wins a match by winning both rounds, it gets 1% more (and thus 3% in total). The four semifinalists get an extra 3%. The total score that a group can get is capped at 15%.

More details about the competition will be announced in due course.